RESEARCH REPORT · 13 NOVEMBER 2026

Spec-driven development and the governed delivery platform

Each approach described on its own terms, then compared fact by fact. Spec-driven development is represented by GitHub Spec Kit, the most widely used open-source toolkit for it.

Summary Part 1: Spec Kit Part 2: The platform Part 3: Comparison What to do Corrections Method and sources

How to read the evidence. Every fact carries one of four labels. Source read directly in the public source code of Spec Kit and three add-ons, cloned on 13 November 2026. Search taken from a web search summary; the original page was not opened. Prototype true of our clickable prototype, which uses demo data and has no back end. Our view our own judgement.

Summary

1.4.0	198	43	0
Spec Kit release of 10 November 2026, MIT licensed	community extensions in its catalog, plus 42 presets	coding agents it can drive	live customers on our platform: it is a prototype

- **Spec Kit is far more capable than our earlier reports said.** It is no longer only "specify, plan, tasks, implement". It now has automated workflows with human review gates, a completeness check, bug-fix and idea-assessment processes, private catalogs, and a large add-on ecosystem. Source
- **Multi-repo and Azure DevOps both exist as community add-ons, not in the core.** One add-on dispatches work to several repositories in dependency order. Another takes an Azure DevOps story to Done with a human approval before every board change. Source
- **The two approaches solve different problems.** Spec Kit is a developer toolkit that lives in the repository and the terminal. Our platform is a governance console for nine business and technical roles, with one approval record across the whole change. Our view

- **Our real advantages are narrower than we claimed.** They are: a screen for non-developers, enforced roles and separation of duties, one audit record, and coverage of integration, validation, release and rollback. Prototype
- **Our real weakness is that nothing is built.** Spec Kit ships today. Our platform is a design with 12 open capabilities. Prototype
- **Independent evidence that written specifications reduce defects is weak.** The largest study found no effect, although it did not test Spec Kit output itself. Search
- **Recommendation: treat Spec Kit as the build engine under the platform, not as a rival.** Our view

PART 1

Spec-driven development with Spec Kit

In spec-driven development the team writes down what to build and why before the AI agent writes code. The specification, plan and task list are Markdown files kept in the repository next to the code.

Facts about the project

Item	Fact	Evidence
Owner and licence	`github/spec-kit`, MIT licence, free	Source
Age	First commit 21 August 2025. Community-led since January 2026	Source
Current release	1.4.0 on 10 November 2026. Three releases in the five days before that	Source
How it is installed	A command-line tool, `specify`. Needs Python 3.11 or later and a supported coding agent	Source
Coding agents supported	43, including Copilot, Claude Code, Codex, Cursor, Gemini and Kiro	Source
Add-ons	198 community extensions, 42 community presets, 4 community bundles, 3 community workflows	Source

Offline use	Documented for air-gapped and firewalled sites, with self-hosted catalogs	Source

How a change moves through it

Step	Command	What it produces
Once per project	`constitution`	The project's standing rules
1	`specify`, then optional `clarify`	`spec.md`: what and why
2	`plan`	`plan.md`: the technical approach
3	`tasks`, then optional `analyze` and `checklist`	`tasks.md`, plus a report of inconsistencies
4	`implement`	Code and tests
5	`converge`	Compares the code with the spec, plan and tasks, and adds any unbuilt work as new tasks. Repeat steps 4 and 5 until it reports "Converged"

All Source. The developer runs each command in the coding agent's chat and reviews the result before the next one.

What it has beyond the basic flow

- **Two more processes.** Bug fixing (assess, fix, test, with a verdict of verified, partial or failed) and idea assessment (ending in go, needs clarification, or kill). Source
- **Workflows.** YAML definitions chain commands, shell tasks and human review gates into runs that can pause and resume. The bundled workflow puts a gate after the specification and after the plan. Source
- **Four ways to customise.** Presets change templates and wording. Extensions add commands and hooks. Workflows automate. Bundles package a setup for a team or role. Source

- **Governed catalogs.** An organisation can host a private catalog of approved add-ons. Microsoft's developer blog recommends giving it priority over the community catalog. Source Search
- **Model Context Protocol server.** Experimental, local only, read-only: four tools that list artifacts and workflows. Source

Multi-repo

Option	What it does	Limits	Evidence
Core	A guide for monorepos: several Spec Kit projects inside one repository	Nothing for separate repositories. No shared or inherited constitution: "you need to maintain that wiring yourself"	Source
`multi-repo` extension, v0.7.1, last commit 11 Nov 2026	One central specs repository. A `workspace.yml` file lists member repositories and their dependencies. Tasks are split per repository and dispatched in dependency order. Each repository runs its own verification commands, and contract checks run after producers. A review gate shows the verification report. A failed run can resume	Its own README: verification "cannot tell a meaningful suite from a trivial one"; rules without a command are "applied by the agent alone"; commands run with the user's privileges	Source
`multi-repo-branching` preset, v1.0.0, last commit April 2026	Finds child repositories and submodules, and adds tasks to create matching feature branches	Branches only. No dispatch or verification	Source
`multi-repo-sync` extension, v1.0.0	Creates the feature branch in affected sub-repositories through hooks	Branches only	Source

A request for first-class multi-repo support in the core (issue 4583) was reported as open. Search

Azure DevOps

The community bundle **Spec Kit for Azure DevOps** (`spec-kit-ado`, catalog version 0.3.2, repository at 0.4.0, last commit 12 November 2026) is the closest thing to our platform in the Spec Kit world. Source

- The developer pulls a story from Azure Boards. The agent writes the spec, plan, code and tests.
- Three approvals are built in: post questions to the story, create the Tasks, open the pull request. The agent shows the exact change and waits for a clear yes.
- Each acceptance criterion is tied to a named test in a per-story checklist, which is attached to the story when it closes.
- Optional sign-offs by a business owner, technical owner and QA. A person writes an approval comment on the story in Azure DevOps, and the comment includes a hash of the exact files signed. Who may sign is read from the target branch, so a feature branch cannot change it.
- It ships Azure Pipelines templates for a gate, a branch guard and a strict verification step.
- Limits it states itself: Azure Boards only (no Jira or GitHub Issues), a personal access token is required, and its checks are "guard rails for a team that reviews its pull requests, not a lock on a machine".

Gaps and risks

Gap	Fact	Evidence
Spec drift	The core has "no dedicated command that automatically reconciles differences" between spec, plan and tasks. `analyze` only reports. `converge` only adds tasks. Reconciliation is left to the team or to community extensions (`sync`, `reconcile`, `refine`)	Source
Approval identity	A core gate is a prompt in a terminal, or a text value passed on the command line. The core documentation describes no approver identity, roles, separation of duties or audit log	Source
Gates can be removed	A community workflow named `yolo` runs the full cycle "without review gates"	Source

Add-on trust	Maintainers "do not review, audit, endorse, or support" community code. Users must vet it. Most add-ons have a single author	Source
Execution safety	A workflow shell step "runs a local command with your privileges. There is no capability sandbox"	Source
Who can use it	Every step runs in a terminal or an agent chat inside a repository. There is no screen for a product owner, release manager or auditor	Source
After the pull request	The core covers nothing after implementation and convergence. Release, deployment and rollback are left to pipelines or community extensions (`ship`, `spec2cloud`)	Source
Process weight	Thoughtworks' review found the tools apply one heavy process to tasks of every size; one summary quotes "a sledgehammer for a nut" on a small bug	Search
Token cost	One informal test measured about 121,000 tokens for Spec Kit against about 58,000 for a lighter tool on the same task	Search
Interfaces between repositories	Critics say each repository can match its own spec while the interface between them still breaks. The `multi-repo` contract checks address part of this	Search Source

Does it improve outcomes?

Study	Finding	How much weight
ESEM 2026 paper (B. Hill): 88,052 pull requests in 119 open-source repositories	Specification quality predicted neither defects ($p = 0.164$) nor rework ($p = 0.860$)	Peer reviewed. But it measured ordinary specification artifacts, mostly links to issues. No repository in the sample used an SDD tool, so Spec Kit itself is untested
Uvik benchmark, 2026: 50 Python tickets	0.46 defects per merged ticket with spec workflows against 0.86 without. Spec Kit merged 40 of 50	Vendor-produced and small. Directional only

OrangeLoops case study, 2026	Code generation was much faster, but stabilising, integration debugging and review still took four days	One project

All Search. We found no independent study that isolates the effect of Spec Kit on defects or delivery time.

PART 2

The Unified Product Delivery Platform

Our platform is a web console in which an AI agent takes a business change request through eight stages, and a named person approves each stage. Everything in this part describes the clickable prototype. It runs in the browser on demo data. No agent, pipeline, database or Azure service is connected. Prototype

Facts about the prototype

Item	Fact
Status	Clickable prototype, 19 screens, React. No back end, no customers, no price
Stages	Plan, Specify, Impact, Design, Build, Integrate, Validate, Release. The order is fixed
Roles	9 roles against 17 permissions in an editable matrix. Platform Admin is locked
Approvals	A named approver per stage, with a named backup. A requester cannot approve their own plan: it goes to a second approver
Documents	Each plan, story, specification, impact assessment, design decision and completion report opens in place, with versions and a SHA-256 fingerprint
Build	A separate Developer Agent in VS Code. The developer copies a prompt from the platform. The platform shows the branch, Boards task, pull request and completion report
Release	Scheduling, freeze windows with exceptions, hold with a required reason, and rollback with a second confirmation
Audit	One log of every decision, access and permission change, with CSV export
Intended architecture	Azure SQL for records, Azure Blob for documents, Azure Boards, Repos and Pipelines
Checks run on the prototype	172 automated click-through checks pass; accessibility scan clean on all 19 screens in light and dark mode

All Prototype. The checks prove the prototype behaves as designed. They say nothing about a production system.

Gaps and risks

Gap	Fact
Nothing is built	Every agent action, pipeline result and document is simulated
One fixed workflow	Eight stages for every change. No fast path for a low-risk change, and no bug-fix path. This is the same "one size" criticism made of spec-driven tools
Copy-and-paste hand-off	The link between the platform and the coding agent is a prompt the developer copies by hand
Multi-repo	The design shows impact across repositories and a pipeline result per repository. The prototype shows this as static data
One ecosystem	Azure DevOps only. No Jira, ServiceNow or GitHub
No model control	No choice of model, no version pinning, no record of which model wrote a document
Evidence is not tamper-proof	Fingerprints are computed, but nothing is stored write-once or signed
No extension model	Stages, templates and checks cannot be changed by the customer
No outcome evidence	No pilot, no measurements

All Prototype. The full list of 12 open capabilities is in `review/panel/STATUS.md`.

PART 3

Comparison

This compares a shipped tool with a design. A row where the platform looks stronger means the design is stronger. Spec Kit's entries are things a team can install today.

Dimension	Spec Kit	Our platform (prototype)	Stronger today
Availability	Released, version 1.4.0, MIT, free Source	Prototype only Prototype	Spec Kit
Where people work	Terminal and agent chat, inside the repository Source	Browser console Prototype	Depends on role
Roles served	Developers. With the Azure DevOps add-on, owners and QA sign off by comment in Azure DevOps Source	9 roles, each with its own view Prototype	Platform (design)
Source of truth	Markdown files in Git Source	A platform record, with documents in Blob storage (intended) Prototype	Spec Kit: it exists and is versioned by Git
Human approval	Core: terminal prompt, no identity. Add-on: hashed sign-off comments by named approvers Source	Named approver per stage, with backup and send-back note Prototype	Platform (design); add-on is close
Separation of duties	Not in core. Add-on reads signers from the target branch Source	Enforced in the role matrix Prototype	Platform (design)
Audit record	Git history, plus a per-story checklist in the add-on Source	One log across all changes, exportable Prototype	Platform (design)

Multi-repo	Community extension: central specs, dependency-ordered dispatch, per-repo verification, contract checks Source	Impact view and per-repo pipeline results, static in the prototype Prototype	Spec Kit
Azure DevOps	Community bundle, human-gated, Boards with Azure Repos or GitHub Source	Designed around it; not connected Prototype	Spec Kit
Jira, Linear, GitHub Issues	Community extensions for each Source	None Prototype	Spec Kit
Choice of coding agent	43 Source	One standalone agent in VS Code Prototype	Spec Kit
Small changes and bugs	Separate bug-fix process; a "lean" preset Source	Same eight stages for everything Prototype	Spec Kit
Spec and code drifting apart	`converge` finds unbuilt work; reconciling documents needs an add-on Source	Not addressed Prototype	Spec Kit
Integration and validation gates	Not in core; community gate extensions Source	Approval screens with pipeline and test results Prototype	Platform (design)
Release, freeze, hold, rollback	Not in core Source	All four, with reasons recorded Prototype	Platform (design)
Customisation	Presets, extensions, workflows, bundles, private catalogs Source	Role matrix and policies only Prototype	Spec Kit
Supply-chain risk	Community add-ons are unaudited and mostly single-author Source	None yet, because nothing is installed Prototype	Neither
Proof of better outcomes	No independent study of the tool Search	None Prototype	Neither

Where Spec Kit is stronger

- It exists, is free, and is released several times a week.
- It works with almost any coding agent and tracker.
- Multi-repo and Azure DevOps already run, through add-ons.
- It can be shaped to a team's process without waiting for a vendor.

Where the platform design is stronger

- People who never open a terminal can read, question and approve.
- Who may approve what is enforced in one place, not per repository.
- One audit record covers the change from request to rollback.
- It covers the stages after the pull request, where a bank's controls sit.

Both panels `Our view`, drawn from the table above.

What to do

- **Stop positioning the platform against Spec Kit.** The claims "no multi-repo", "no gates" and "stops at implementation" are out of date and a technical buyer will check them.
- **Position the platform as the governance layer above a spec-driven engine.** The Developer Agent step could run Spec Kit in the repository, and the platform could hold the approvals, roles and audit record.
- **Pilot `spec-kit-ado` on one real story.** It is the cheapest way to learn how far free tooling already goes for an Azure DevOps bank, and what is still missing for auditors and business approvers.
- **Fix the weaknesses we share.** Add a fast path for low-risk changes and a bug-fix path, so the platform is not open to the same "too heavy" criticism.
- **Replace copy-and-paste with a real connection.** Spec Kit's workflow runs, verdict inputs and experimental MCP server are the natural points to connect to.
- **Vet before use.** Any community add-on needs a code review and a pinned version in a private catalog before it touches a bank's repositories.

All `Our view`.

Corrections to our earlier reports

We said	What the source shows
Spec Kit is not multi-repo; teams use workarounds	A maintained community extension does central specs, ordered dispatch and per-repo verification. The core has monorepo guidance only
Spec Kit stops at implementation	The core now adds `converge`, bug fixing and idea assessment. Release is still outside the core
Spec Kit has no approval gates	Workflows have human review gates. The Azure DevOps add-on adds named, hashed sign-offs
We are ahead on Azure DevOps	A community bundle already connects Boards, Repos and Pipelines with human approval. Ours is not connected

The affected files are `review/panel/compare.md`, `review/panel/market-agentic-sdlc.md`, and the stakeholder report and video of 12 November.

Method and sources

- **Source reading.** We cloned four public repositories on 13 November 2026 and read their documentation, catalogs and templates. Counts come from the catalog files. We did not install or run Spec Kit, so behaviour is as documented, not as observed.
- **Search.** Studies and commentary come from web search summaries. We could not open the original pages. Check any figure before quoting it outside the team.
- **Not checked.** GitHub stars, download numbers, the current state of issue 4583, and the quality or maintenance of add-ons other than the three we read.
- **Kiro and other spec-driven tools** are out of scope here. They are covered in the earlier market report.

Sources read in full

- github/spec-kit, commit 2ee2b1e of 12 November 2026: README, CHANGELOG, `docs/`, `templates/`, and all catalog files
- joaopaschuino/spec-kit-multirepo: README
- sakitA/spec-kit-preset-multi-repo-branching: README

- praveenlavu/spec-kit-ado: README and `rules/`

Sources from search summaries

- Hill, "Specification Artifacts in Open-Source Pull Request Workflows Do Not Reduce Defects", ESEM 2026
- Uvik, Spec-Driven Development Benchmark 2026
- OrangeLoops, Spec Kit and Claude Code case study
- Böckeler, "Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl"
- Microsoft, "From Spec-First to Enterprise-Ready: Extending GitHub Spec Kit"
- "Spec Driven Development Is Wasting Tokens" (token comparison)
- "Spec-driven development has a blind spot: the seams"
- Spec Kit issue 4583, multi-repo support